

Software Development Lifecycle: four fundamental phases shared by all software engineering projects, though different projects may differ in their emphasis of each phase. A process of gradual refinement.

why
Planning
 identify system's business value
 project sponsor → system request
 (sponsor + IS Department → feasibility analysis)
 approval committee
 ↓
 project management → work plan
 → project plan

who, what, where, when
Analysis
 analysis strategy: current problems → new system
 requirements gathering
 analysis models - how business will use new system
system proposal - an analysis and high-level design

Implementation
 system construction/design
 custom built, pre-built, outsourced
 coding style, documentation
 coordination
 language
 testing
 risk? cost? time?
 installation (direct, parallel, single location, phased)
 transition to new system, integrating
 API data integration
 training plan, user guides
 support plan
 maintenance?

successful implementation: use and satisfaction
 management support, user demographics, performance,

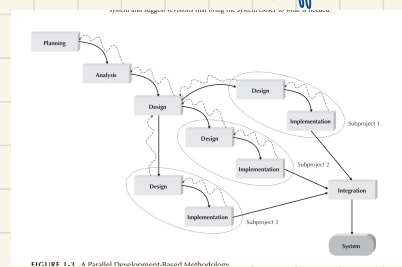
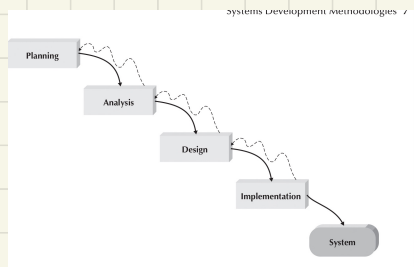
how
Design (OOB)
 architecture design (hardware, software, network)
 interface design (user movements, forms, reports)
 database and file design (what data, where stored)
 program design (which programs, program goals)
 } system specification

presentation
 Human Computer Interaction
 boundary UI classes
 UI design, Java UI
 task management
 exception handling
 database
 broker
 detailed class design
 operations
 (CRUD)
 associations
 multiplicity
 minimise
 collection classes

Good design:
 class has single focused purpose.

easier
 reusable
 • high cohesion
 internal functional relatedness
 • low coupling
 less ripple effect
 easier maintenance
 encapsulation

tradeoff based on context: efficiency
 usability
 accessibility
 security



Process Methodologies

1. Structured formal step-by-step, strong documentation

Waterfall sequential progression
long key deliverables
difficult to go backwards

clearly defined requirements
minimises changes throughout project

long time between Planning → Implementation

- missed requirements are costly - business environment changes

parallel general design → subprojects implemented in parallel
↳ integrated → delivered

reduces time to delivery

subprojects are *not* independent

- changes lead to difficult integration

2. Rapid Application Development (RAD) get system to users quickly, revise, repeat

- CASE tools • visual programming languages • code generators

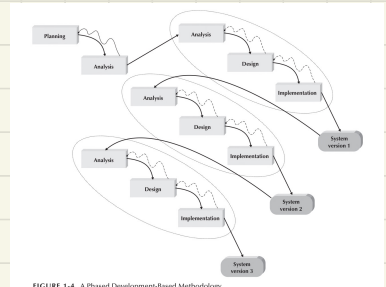
Phased Development prioritised versions

Version 1 + feedback → Version 2...

gets useful system to users

identifies additional requirements

users experience incomplete system



Prototyping analysis, design, implementation, concurrently, repeated

basic analysis and design, quick-and-dirty minimal implementation

very quickly provides a system, gathers feedback

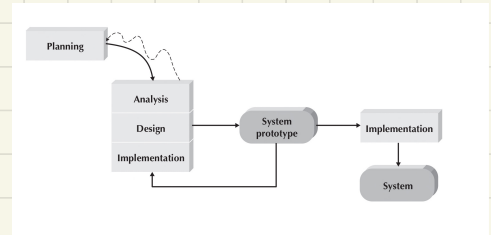
reassures users

refine real requirements

fast-pace limits methodical analysis

big changes - initial design choices cause problems

fundamental issues not recognised until later



Throwaway Prototyping more thorough analysis phase

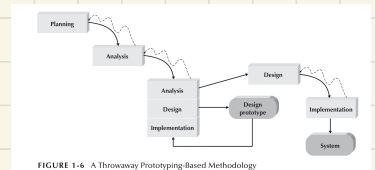
design-prototype to identify issues with underlying requested features

resolved issues → Design + Implementation

advantages of in-depth analysis + prototype refinement of issues

more stable and reliable systems

longer development

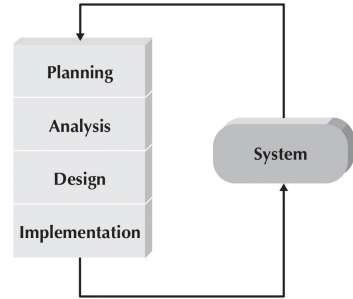


3. Agile Development focus on developers, software, customers

v.s.

development processes, plans, documentation

- embrace changing requirements
- progress measured by working software
- technical excellence and good design
- customers and developers work together (face-to-face)
- object-oriented approach
- provide tools and environment and individuals deliver
- self-organising teams
- sustainable pace
- regular reflection



Extreme Programming

rapid feedback KISS embrace change quality-first
communication, simplicity, feedback, courage

- continuous testing (daily, error-free)
- pair programming (shared responsibility, developer improvement)
- end-users (user stories)
- large projects, or team not jelled → outcome doubtful (collaboration, outsourcing...)
- discipline required or else too chaotic
- lack of documentation - difficult to maintain large system
- on-site user input.

Scrum

self-organised/directed teams - no team leader, symbiotic
sprint requirements, daily scrum meetings.
sprint results → user → next sprint.
"bad" decision > block, issues identified at next meeting.

communication, developer improvement (Scrum meetings)

incremental requirements per sprint

iterative development

scale up? 'scrum of scrums'

Ability to Develop Systems	Structured Methodologies		RAD Methodologies		Agile Methodologies		
	Waterfall	Parallel	Phased	Prototyping	Throwaway Prototyping	XP	SCRUM
With Unclear User Requirements	Poor	Poor	Good	Excellent	Excellent	Excellent	Excellent
With Unfamiliar Technology	Poor	Poor	Good	Poor	Excellent	Good	Good
That Are Complex	Good	Good	Good	Poor	Excellent	Good	Good
That Are Reliable	Good	Good	Good	Poor	Excellent	Excellent	Excellent
With a Short Time Schedule	Poor	Good	Excellent	Excellent	Good	Excellent	Excellent
With Schedule Visibility	Poor	Poor	Excellent	Excellent	Good	Excellent	Excellent

FIGURE 1-8 Criteria for Selecting a Methodology

Quality (Cross-lifecycle activity)

Prevent: cost of rework,
returns, complaints, helpline/warranty

Product

Validation

(functional)

Build correct product?

conformance to requirements
↳ support
↳ customer satisfaction

Metric: software design: coupling
cohesion

testing Maintenance:
perfective, corrective, adaptive, preventive

attributes, components, capabilities

Testing:
- test planning
- identify test cases/scenarios from use cases
- record results
- evaluate test progress

Functional

Unit testing

(packaging, boundary values, decision tables, state transition)

Integration testing

System testing

Acceptance testing

Non-Functional

Performance Testing

Security Testing

Usability Testing

bug report, test log.

Static code reviews

Dynamic

Structural white box

Behavioural black box

Manual vs. Automated

JUnit

Selenium

verify software meets requirements

identify differences between expected and actual results

cannot guarantee correctness

difficult to reproduce errors

complexity, time consuming, exhaustive not possible

McCall's Quality Factors

Revision	Transition	Operation
Maintainability	Portability	Correctness
Flexibility	Reusability	Reliability
Testability	Interoperability	Efficiency
		Integrity
		Usability

Process

Verification

(non functional)

Build product correctly?

quality mindset, standards

thorough testing

installation and managing changes (github)

(team progression)

Metric: process maturity

CMMI

ISO 9001:2015 (certification)

Capability Maturity Model:

1. Initial
2. Managed
3. Defined
4. Quantitatively Managed
5. Optimizing

ISO:

1. Customer Focus
2. Leadership
3. Engagement of People
4. Process Approach
5. Continual Improvement
6. Evidence based Decision Making
7. Relationship Management.

certification ensures correct procedures are in place
external credibility
accredited market

Project Management (Planning / Cross-lifecycle)

PMBOK:

Integration

Scope

Time

Cost

Quality

Human Resource

Communications

Risk

Procurement

Phases:

Initiating

Planning

Executing

Monitoring and Controlling

Closing

project manager, stakeholders, project plan - feasibility (technical, economic, organizational)

team contract, scope statement, schedule, risk charter

leadership skills, milestone report

performance reports, requested changes, plan updates

final report

Competing demands

scope, time, cost, quality

stakeholders needs and expectations

identified unidentified

Success

meet requirements

on time

on budget

Project Management Plan:

Goals / scope

all / only the required work

Organisation and responsibilities

estimation

Schedules and resource requirements

limits

Communications activities

decompose problem

analyse risks, function points, process tasks, use-cases

Loc: errors per KLOC

6

doc pages

errors

people
problem
products
resources

inputs

outputs

files

external

interactions

reconcile estimations

Risk Management:

project schedule / resources

products quality / performance

business organisation

Identification

Analysis likelihood, consequences

Procurement

Planning avoid / minimise effects

Contingency

Monitoring change in risk probability

Alternative

Insurance

Risk Information Sheet

PRINCE

Diagrams

Behavioural

- Use Case** capture the Use Cases and relationships among Actors of the system. Describes the functional requirements of the system, how external operators interact at the system boundary and the response of the system
- Process Model)**
Activity model the behaviours of the system, how these behaviours are related to the flow of the system
- State Transition** illustrate how an element can move between states, classifying its behaviour according to transition triggers and constraining guards.
- Interaction (Sequence)** structured representation of the behaviour of several objects within a single use case, use case realisation. Objects, messages, focus of control, triggers

Structural

- (Class Model)**
Class depicts the static design of the object-oriented planned software, providing information about the classes, their attributes and operations and the relationships between them.

Use Case Description Template

Use Case

Context of Use

Scope Business, System

Level Subroutine, User Goal, Summary

Primary Actors

Stakeholders/Interest

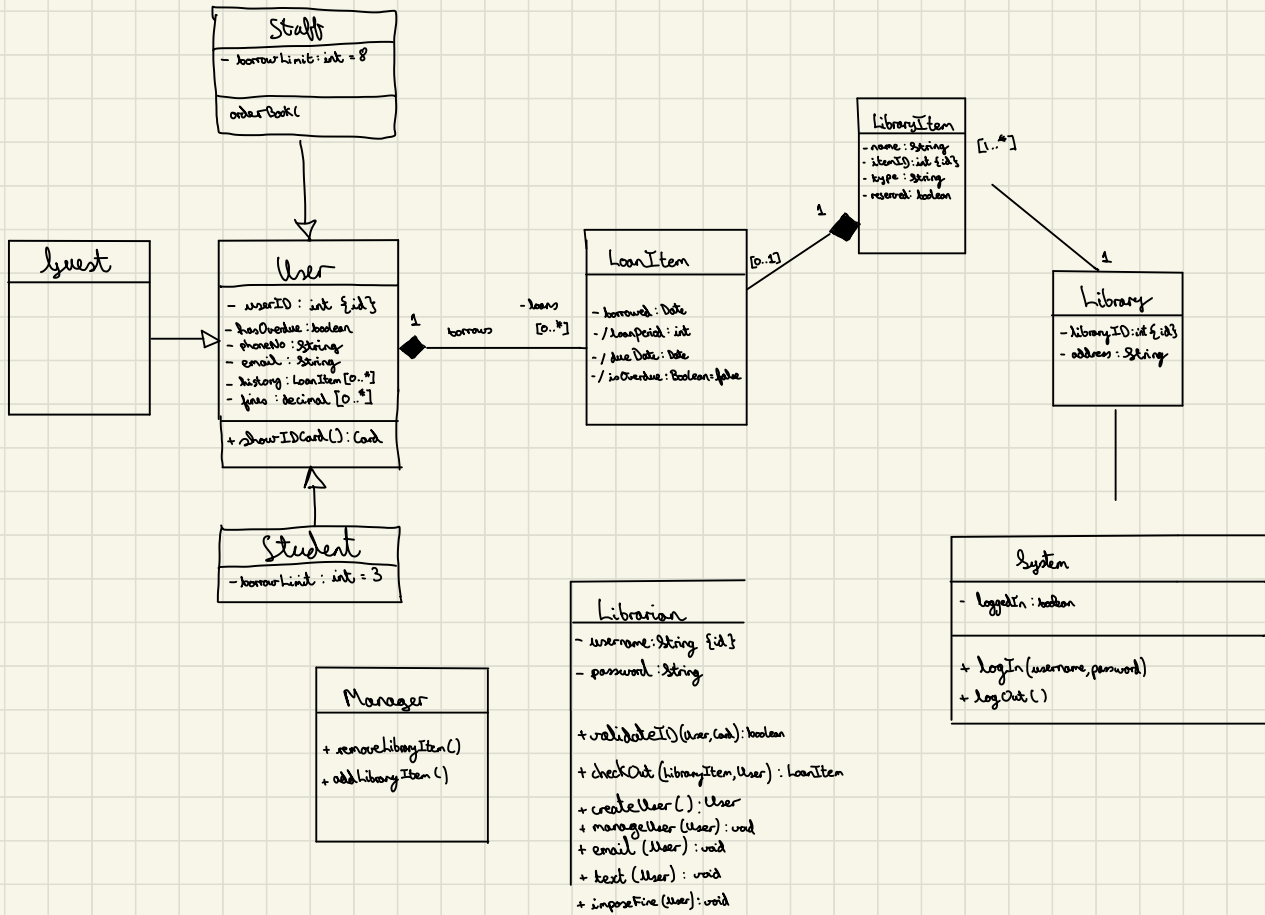
Preconditions

Minimal Guarantees

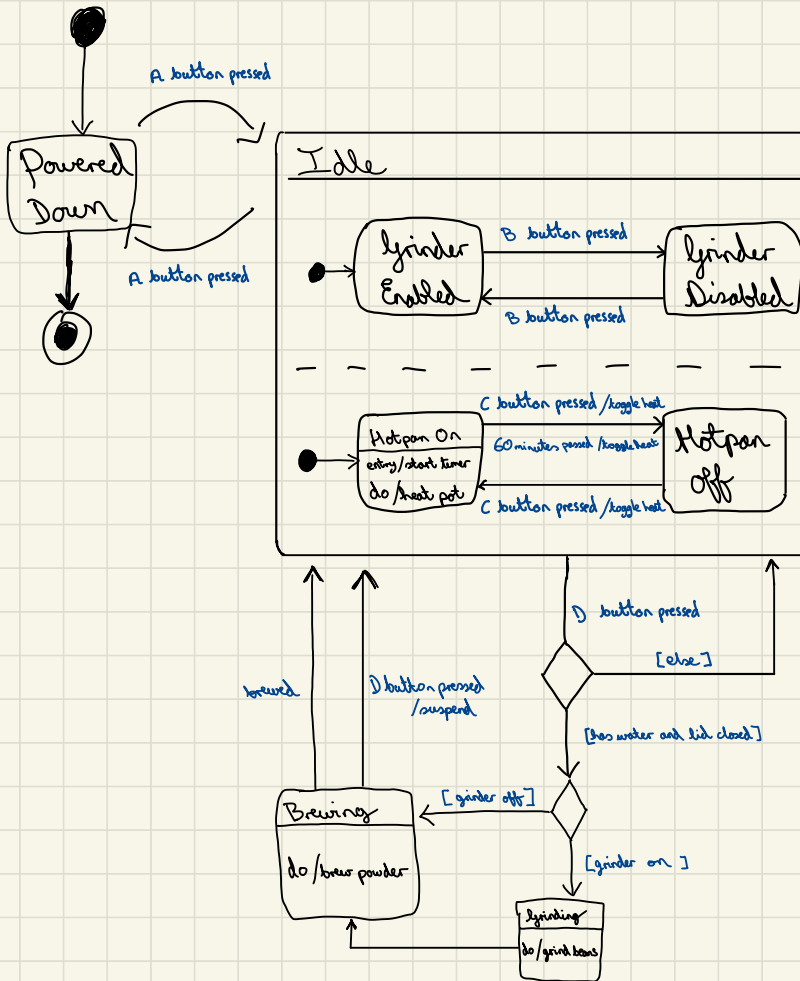
Success Guarantees

Description

Extension.



18/19
Q.3



19/10 Q.3

